

MÓDULO I · INGENIERÍA DE IA

# 03

CAPÍTULO

---

*Retrieval-Augmented Generation: cuando  
el conocimiento del modelo no alcanza*

# Retrieval-Augmented Generation: cuando el conocimiento del modelo no alcanza

---

*"La pregunta más importante no es cuánto sabe un modelo, sino cómo accede al conocimiento que necesita en el momento adecuado."*

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender el problema que dio origen a Retrieval-Augmented Generation (RAG).
  - Diferenciar conocimiento entrenado de conocimiento externo.
  - Identificar las limitaciones de un LLM trabajando de forma aislada.
  - Entender por qué RAG representa una decisión arquitectónica y no un nuevo modelo.
- 

## Introducción

---

En los capítulos anteriores analizamos cómo aprende un modelo de lenguaje, cómo representa la información y cómo genera respuestas.

También vimos que un LLM posee una limitación fundamental.

Su conocimiento está determinado por dos elementos:

- los parámetros obtenidos durante el entrenamiento;
- el contexto disponible durante la inferencia.

Esto plantea un desafío inmediato para cualquier organización.

¿Cómo responder preguntas sobre información que nunca formó parte del entrenamiento del modelo?

---

## Un problema cotidiano

---

Imaginemos una empresa que desea construir un asistente para responder consultas sobre:

- procedimientos internos;
- contratos;
- manuales técnicos;
- expedientes;
- políticas de seguridad;
- documentación de proyectos.

Ninguno de esos documentos fue utilizado durante el entrenamiento del modelo.

Aunque el LLM sea extremadamente capaz, simplemente no puede conocer información privada que nunca estuvo disponible públicamente.

Entrenar nuevamente el modelo completo tampoco resulta una alternativa razonable.

El costo económico, el tiempo requerido y la complejidad técnica hacen que esta estrategia sea inviable para la mayoría de las organizaciones.

---

## Las alternativas tradicionales

---

Antes de la aparición de RAG existían dos caminos habituales.

El primero consistía en confiar únicamente en el conocimiento general del modelo.

Esto producía respuestas fluidas, pero incapaces de incorporar información específica de la organización.

El segundo consistía en volver a entrenar el modelo con documentación propia.

Aunque posible en determinados escenarios, esta estrategia presenta importantes desventajas:

- alto costo computacional;
- largos tiempos de entrenamiento;
- dificultad para actualizar información;
- necesidad de infraestructura especializada;
- riesgo de degradar capacidades previamente adquiridas.

Era evidente que hacía falta un enfoque diferente.

---

## Una idea sencilla con enorme impacto

---

La propuesta detrás de Retrieval-Augmented Generation puede resumirse en una frase.

*En lugar de enseñar permanentemente toda la información al modelo, recuperemos únicamente la información necesaria para responder la consulta actual.*

Esta idea cambia completamente la arquitectura.

El conocimiento deja de almacenarse exclusivamente dentro de los parámetros del modelo.

Parte del conocimiento permanece en repositorios externos y solo se incorpora cuando resulta necesario.

---

## Un cambio de paradigma

---

Con RAG el modelo deja de ser la única fuente de conocimiento.

La arquitectura pasa a estar formada por varios componentes que colaboran entre sí.

El modelo continúa siendo responsable de comprender la consulta y generar la respuesta.

Sin embargo, la recuperación de información queda delegada a un mecanismo especializado.

Esta separación de responsabilidades aporta múltiples beneficios:

- actualización inmediata de documentos;
- reducción de costos;
- mayor trazabilidad;
- respuestas fundamentadas en fuentes concretas;
- menor necesidad de reentrenamiento.

## Caso real

---

Una organización pública mantiene miles de resoluciones administrativas que cambian periódicamente.

Cada semana se incorporan nuevas versiones y se derogan procedimientos anteriores.

Entrenar un modelo completo después de cada modificación sería impracticable.

Con una arquitectura RAG, las resoluciones permanecen en un repositorio documental.

Cuando un usuario realiza una consulta, el sistema recupera únicamente los documentos relevantes y los incorpora al contexto del modelo.

La actualización ocurre sobre los documentos, no sobre el LLM.

---

## Ideas clave

---

- Un LLM no conoce automáticamente la información privada de una organización.
- Reentrenar un modelo rara vez es la mejor solución.
- RAG separa el conocimiento del modelo del conocimiento documental.
- Retrieval-Augmented Generation es una arquitectura, no un modelo nuevo.

---

## Próxima sección

---

En la siguiente sección analizaremos la arquitectura completa de un sistema RAG y seguiremos el recorrido de una consulta desde que el usuario formula una pregunta hasta que el modelo genera una respuesta respaldada por información recuperada.

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

# Arquitectura de un sistema RAG

---

*"RAG no es una llamada a un modelo. Es una arquitectura compuesta por múltiples componentes especializados que colaboran para producir una respuesta confiable."*

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender los componentes principales de una arquitectura RAG.
  - Seguir el recorrido completo de una consulta.
  - Diferenciar recuperación de información y generación de respuestas.
  - Identificar los puntos donde un arquitecto puede optimizar el sistema.
- 

## Introducción

---

Una vez comprendido el problema que intenta resolver RAG, el siguiente paso consiste en analizar cómo funciona internamente.

A diferencia de un chatbot tradicional, un sistema RAG incorpora un flujo adicional antes de invocar al modelo de lenguaje.

Ese flujo tiene un único objetivo: recuperar el conocimiento más relevante para responder la consulta del usuario.

---

## Componentes principales

---

Una arquitectura RAG típica está compuesta por los siguientes elementos:

- Cliente o aplicación consumidora.
- Motor de orquestación.
- Modelo de embeddings.
- Base de datos vectorial.

- Repositorio documental.
- Large Language Model.
- Sistema de observabilidad y monitoreo.

Cada componente cumple una responsabilidad específica.

Separar responsabilidades facilita la evolución, el mantenimiento y la escalabilidad.

---

## Flujo de una consulta

---

Cuando un usuario realiza una pregunta, el proceso puede resumirse así:

1. La aplicación recibe la consulta.
2. La consulta se convierte en un embedding.
3. Ese embedding se utiliza para buscar documentos similares en la base vectorial.
4. Se recuperan únicamente los fragmentos más relevantes.
5. Los fragmentos recuperados se incorporan al contexto del modelo.
6. El LLM genera una respuesta fundamentada en ese contexto.
7. La respuesta se devuelve al usuario.



---

## ¿Por qué separar recuperación y generación?

---

Un error frecuente consiste en pensar que el modelo "busca" documentos.

En realidad, la búsqueda suele estar a cargo de un componente independiente.

El modelo recibe únicamente el resultado de esa búsqueda.

Esta separación aporta varias ventajas:

- permite cambiar el modelo sin modificar el repositorio documental;
- facilita optimizar la recuperación de información;

- reduce costos de inferencia;
- mejora la trazabilidad de las respuestas.

Cada componente puede evolucionar de forma independiente.

---

## Arquitectura antes que modelo

---

En proyectos reales es habitual obtener mayores mejoras optimizando la recuperación que reemplazando el LLM.

Una mala estrategia de recuperación puede enviar contexto irrelevante.

Un contexto irrelevante conduce a respuestas incorrectas, incluso utilizando un modelo de última generación.

Por el contrario, una recuperación precisa permite que modelos más pequeños produzcan resultados sorprendentes.

Esta es una de las razones por las cuales RAG se considera un patrón arquitectónico.

---

## Caso de estudio

---

Dos organizaciones utilizan exactamente el mismo modelo fundacional.

La primera envía manuales completos al modelo en cada consulta.

La segunda recupera únicamente los cinco fragmentos más relacionados mediante búsqueda semántica.

Aunque ambas utilizan el mismo LLM, la segunda obtiene menor latencia, menor costo y respuestas más precisas.

La diferencia no reside en el modelo.

Reside en la arquitectura.

---



## Ideas clave

---

- RAG incorpora una etapa de recuperación previa a la generación.
  - La base vectorial y el LLM cumplen responsabilidades distintas.
  - La calidad de la recuperación condiciona la calidad de la respuesta.
  - La arquitectura completa determina el desempeño del sistema.
- 

## Próxima sección

---

En la siguiente sección estudiaremos cómo se preparan los documentos antes de ingresar a una arquitectura RAG, incluyendo particionado (*chunking*), normalización y generación de embeddings.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

---

## Preparación de documentos: la base de un RAG de calidad

---

*"La calidad de un sistema RAG comienza mucho antes de la primera consulta. Comienza en la preparación del conocimiento."*

---

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender por qué la preparación documental es una etapa crítica.
- Entender el concepto de *chunking* y su impacto en la recuperación.
- Conocer el rol de los metadatos dentro de una arquitectura RAG.
- Identificar buenas prácticas para construir una base documental escalable.

---

## Introducción

---

Cuando se observa un sistema RAG funcionando, suele parecer que todo comienza con la pregunta del usuario.

En realidad, una parte sustancial del trabajo ocurre mucho antes.

Antes de responder una sola consulta es necesario transformar el conocimiento de la organización en un formato que pueda ser recuperado eficientemente.

Esta etapa recibe distintos nombres según la herramienta utilizada, pero conceptualmente constituye el proceso de **ingestión y preparación documental**.

La calidad de este proceso condiciona directamente la calidad de las respuestas futuras.

---

## La diversidad de las fuentes

---

En un entorno empresarial la información rara vez se encuentra organizada de manera uniforme.

Es habitual encontrar:

- documentos PDF;
- manuales técnicos;
- archivos de Office;
- páginas wiki;
- correos electrónicos;
- bases de conocimiento;
- registros de sistemas;
- documentación en Markdown;
- código fuente;
- procedimientos internos.

Cada formato requiere mecanismos específicos de extracción y normalización.

Antes de pensar en modelos, el arquitecto debe garantizar que el contenido pueda ser procesado de forma consistente.

---

## Normalización

---

La normalización consiste en convertir documentos heterogéneos en una representación uniforme.

Dependiendo del proyecto, esto puede implicar:

- eliminar encabezados y pies de página repetitivos;
- corregir errores de codificación;
- unificar formatos de fecha;
- preservar tablas relevantes;
- eliminar contenido duplicado;
- extraer texto de documentos escaneados mediante OCR.

No se trata de "limpiar texto".

Se trata de preservar el conocimiento realmente útil para el sistema.

---

## El problema del tamaño

---

Una vez normalizados los documentos aparece un nuevo desafío.

Los modelos de lenguaje trabajan con ventanas de contexto limitadas.

Enviar documentos completos rara vez constituye una buena estrategia.

Por ello surge el concepto de **chunking**.

---

## ¿Qué es el chunking?

---

El *chunking* consiste en dividir un documento en fragmentos más pequeños denominados **chunks**.

Cada fragmento debe conservar suficiente contexto para mantener su significado, pero también debe ser lo bastante compacto como para recuperarse eficientemente.

El objetivo no es partir un archivo cada determinada cantidad de caracteres.

El objetivo es preservar unidades de conocimiento coherentes.

Por ejemplo:

- una sección de un manual;
- un procedimiento completo;
- un artículo de una normativa;
- una función documentada;
- un apartado de una política interna.

---

## ¿Por qué el chunking es tan importante?

---

Supongamos un procedimiento compuesto por diez pasos.

Si el algoritmo divide el texto exactamente en la mitad, es posible que los primeros cinco pasos queden en un fragmento y los restantes en otro.

Una consulta relacionada con el procedimiento podría recuperar únicamente la primera mitad.

La respuesta sería técnicamente correcta, pero estaría incompleta.

Este ejemplo muestra que el chunking no es un problema de longitud.

Es un problema de diseño del conocimiento.

---

## El papel de los metadatos

---

Cada fragmento puede enriquecerse con información adicional.

Estos datos reciben el nombre de **metadatos**.

Algunos ejemplos son:

- autor;
- fecha de creación;
- versión;
- área responsable;
- tipo de documento;

- clasificación de seguridad;
- idioma;
- sistema de origen.

Los metadatos permiten aplicar filtros antes o después de la búsqueda semántica.

Por ejemplo:

- recuperar únicamente documentos vigentes;
- limitar resultados a una determinada unidad organizativa;
- excluir versiones obsoletas;
- restringir información confidencial.

Un sistema RAG empresarial rara vez funciona únicamente con similitud vectorial.

Los metadatos forman parte de la estrategia de recuperación.

---

## Pipeline de preparación documental

---

Una arquitectura típica de ingestión puede representarse de la siguiente manera:



Obsérvese que el modelo de lenguaje todavía no participa.

Todo este trabajo ocurre antes de la primera consulta.

---

## Caso de estudio

---

Una empresa incorpora diez años de documentación técnica a su plataforma RAG.

Durante las pruebas iniciales las respuestas son inconsistentes.

El problema no reside en el modelo ni en la base vectorial.

Los documentos fueron divididos cada mil caracteres sin respetar títulos, subtítulos ni límites de sección.

Al redefinir el chunking utilizando la estructura lógica de los documentos y agregando metadatos sobre versión y área responsable, la precisión mejora de forma significativa sin modificar el LLM.

La mejora provino de la arquitectura de ingestión.

---

## Buenas prácticas

---

Al diseñar el proceso de preparación documental conviene considerar los siguientes principios:

- preservar el significado de cada fragmento;
- evitar duplicados innecesarios;
- mantener trazabilidad hacia el documento original;
- enriquecer los chunks con metadatos útiles;
- automatizar la actualización del repositorio;
- versionar el conocimiento cuando corresponda.

Estas prácticas facilitan el mantenimiento del sistema y mejoran la calidad de la recuperación.

---

## Ideas clave

---

- Un sistema RAG comienza con la preparación del conocimiento.
- El chunking determina qué información podrá recuperarse posteriormente.
- Los metadatos complementan la búsqueda semántica.
- La calidad de la ingestión condiciona la calidad de todo el sistema.

---

## Próxima sección

---

En la siguiente sección estudiaremos cómo se generan los embeddings para cada fragmento y por qué la elección del modelo de embeddings puede tener tanto impacto como la elección del propio LLM.

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

## Modelos de Embeddings: el motor silencioso de un sistema RAG

*"En un sistema RAG, la calidad de la recuperación depende mucho más de los embeddings de lo que la mayoría de los equipos imagina."*

### Objetivos de aprendizaje

Al finalizar esta sección podrás:

- Comprender qué hace un modelo de embeddings.
- Diferenciar un modelo de embeddings de un LLM.
- Conocer los principales criterios para seleccionar un modelo de embeddings.
- Entender el impacto de esa elección sobre la precisión del sistema.

## Introducción

Cuando se habla de Inteligencia Artificial generativa, casi toda la atención se concentra en el Large Language Model.

Sin embargo, en una arquitectura RAG existe otro componente igualmente importante: el modelo encargado de generar los embeddings.

Si los embeddings representan incorrectamente el significado de los documentos, la búsqueda semántica devolverá resultados poco relevantes y el LLM responderá utilizando un contexto deficiente.

En consecuencia, una mala recuperación rara vez puede compensarse con un mejor modelo generativo.

---

## ¿Qué hace un modelo de embeddings?

---

Su función consiste en transformar un contenido en un vector numérico que preserve, en la mayor medida posible, su significado.

Ese contenido puede ser:

- una palabra;
- una oración;
- un párrafo;
- un documento completo;
- código fuente;
- imágenes o audio, dependiendo del modelo.

Una vez generado el vector, la comparación entre documentos deja de depender de coincidencias textuales y pasa a basarse en cercanía semántica.

---

## Un modelo diferente al LLM

---

Aunque ambos trabajan con lenguaje, cumplen objetivos distintos.

---

Componente Responsabilidad principal

---

Modelo de Embeddings Representar significado mediante vectores.

**Large Language Model Comprender el contexto y generar una respuesta.**

---

En muchos proyectos ambos modelos son completamente diferentes.

Incluso pueden provenir de proveedores distintos.

Esta separación permite optimizar costos y rendimiento sin afectar toda la arquitectura.

---



## ¿Cómo elegir un modelo?

---

No existe un modelo universalmente mejor.

La elección depende del problema.

Algunos criterios habituales son:

- idioma soportado;
- dominio del conocimiento;
- longitud máxima del texto;
- precisión en búsqueda semántica;
- velocidad de inferencia;
- consumo de memoria;
- costo de operación;
- posibilidad de ejecutarlo localmente.

Un arquitecto evalúa estos factores antes de seleccionar una tecnología.

---

## Reentrenar o reutilizar

---

En la mayoría de las implementaciones empresariales no resulta necesario entrenar un modelo de embeddings desde cero.

Es mucho más habitual reutilizar modelos previamente entrenados y concentrar el esfuerzo en la calidad de los documentos, el *chunking* y la estrategia de recuperación.

Solo en dominios altamente especializados suele justificarse un entrenamiento o ajuste adicional.

---

## Caso de estudio

---

Una organización dispone de dos sistemas RAG.

Ambos utilizan exactamente el mismo LLM.

El primero genera embeddings con un modelo optimizado para inglés.

El segundo utiliza un modelo entrenado específicamente para español.

Las consultas realizadas en español muestran diferencias significativas en la recuperación de documentos.

El cambio no estuvo en el modelo generativo.

Estuvo en la representación semántica utilizada para indexar el conocimiento.

---

## Buenas prácticas

---

- Utilizar el mismo modelo de embeddings para indexación y búsqueda.
  - Evitar mezclar espacios vectoriales incompatibles.
  - Evaluar la calidad mediante conjuntos de consultas reales.
  - Medir precisión antes de reemplazar un modelo.
  - Versionar el proceso de indexación cuando cambie el modelo de embeddings.
- 

## Ideas clave

---

- El modelo de embeddings determina la calidad de la recuperación.
  - Embeddings y LLM cumplen funciones diferentes.
  - Elegir correctamente el modelo de embeddings puede producir mejoras superiores a cambiar de LLM.
  - La evaluación debe realizarse con datos representativos del dominio.
- 

## Próxima sección

---

En la siguiente sección estudiaremos las bases de datos vectoriales, cómo almacenan millones de embeddings y cómo realizan búsquedas eficientes sobre espacios de alta dimensión.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

---

# Bases de datos vectoriales: donde vive el conocimiento recuperable

---

*"Un sistema RAG no busca documentos. Busca proximidad matemática entre representaciones semánticas."*

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender qué es una base de datos vectorial.
  - Entender por qué una base relacional tradicional no resuelve eficientemente este problema.
  - Conocer los principios de búsqueda aproximada por vecinos más cercanos (ANN).
  - Incorporar criterios arquitectónicos para seleccionar una tecnología vectorial.
- 

## Introducción

---

Una vez preparados los documentos y generados sus embeddings, surge una nueva pregunta.

¿Dónde almacenarlos?

Podríamos guardar cada vector en una tabla relacional.

Sin embargo, cuando el repositorio contiene cientos de miles o millones de documentos, buscar el vector más parecido deja de ser un problema de almacenamiento y pasa a ser un problema de geometría computacional.

Las bases de datos vectoriales nacieron para resolver precisamente ese desafío.

---

## Del SQL a la similitud

---

Las bases de datos relacionales fueron diseñadas para responder preguntas como:

- ¿Cuál es el cliente con este identificador?

- ¿Qué facturas pertenecen a este período?
- ¿Qué pedidos superan determinado importe?

Todas ellas pueden responderse mediante igualdad, rangos o índices clásicos.

En cambio, un sistema RAG necesita responder otra pregunta:

*¿Qué documentos son semánticamente más parecidos a esta consulta?*

Aquí ya no buscamos igualdad.

Buscamos cercanía en un espacio vectorial.

---

## ¿Qué almacena una base vectorial?

---

Cada registro suele contener:

- el embedding;
- el identificador del documento;
- el texto o referencia al contenido;
- metadatos;
- información necesaria para indexación.

El embedding constituye el elemento central.

Los demás datos permiten recuperar y contextualizar el contenido original.

---

## Búsqueda por similitud

---

Comparar un vector contra millones de registros calculando todas las distancias sería demasiado costoso.

Por ello la mayoría de las bases vectoriales implementa algoritmos de **Approximate Nearest Neighbor (ANN)**.

Estos algoritmos sacrifican una pequeña cantidad de precisión para obtener mejoras muy significativas en tiempo de respuesta.

En aplicaciones empresariales este intercambio suele resultar aceptable.

---

## Índices especializados

---

Al igual que una base relacional utiliza índices B-Tree o Hash, las bases vectoriales emplean estructuras específicas para espacios de alta dimensión.

Entre las más utilizadas se encuentran:

- HNSW (Hierarchical Navigable Small World);
- IVF (Inverted File Index);
- PQ (Product Quantization).

Como arquitecto no es imprescindible memorizar cada algoritmo, pero sí comprender que la estrategia de indexación influye directamente sobre:

- latencia;
- consumo de memoria;
- precisión de la recuperación;
- costo operativo.

---

## Tecnologías disponibles

---

Actualmente existen múltiples alternativas.

Algunas de las más utilizadas son:

- **FAISS**, biblioteca optimizada para búsquedas vectoriales locales.
- **Qdrant**, base vectorial orientada a aplicaciones modernas.
- **Milvus**, diseñada para grandes volúmenes de datos.
- **Weaviate**, con capacidades adicionales de búsqueda híbrida.
- **pgvector**, extensión para PostgreSQL que incorpora almacenamiento y búsqueda vectorial.

La decisión depende del contexto del proyecto.

No existe una opción universalmente superior.

---

## Criterios de selección

---

Antes de elegir una tecnología conviene analizar:

- volumen esperado de documentos;
- frecuencia de actualización;
- necesidad de alta disponibilidad;
- integración con la infraestructura existente;
- costos de operación;
- experiencia del equipo;
- requerimientos regulatorios.

En muchos proyectos pequeños, una base relacional con soporte vectorial puede ser suficiente.

En plataformas corporativas de gran escala suele justificarse una solución especializada.

---

## Caso de estudio

---

Una organización comienza con cincuenta mil documentos utilizando PostgreSQL y una extensión vectorial.

Dos años después supera los treinta millones de fragmentos indexados.

Las búsquedas siguen siendo correctas, pero la latencia deja de cumplir los objetivos del negocio.

El problema no radica en el modelo de IA.

La arquitectura de almacenamiento ya no resulta adecuada para la nueva escala.

La migración hacia una base vectorial especializada se convierte en una decisión arquitectónica, no funcional.

---

## Ideas clave

---

- Una base vectorial almacena representaciones semánticas, no únicamente texto.
- La búsqueda por similitud requiere algoritmos diferentes a los utilizados por SQL tradicional.
- La estrategia de indexación influye en rendimiento y precisión.

- La tecnología elegida debe responder a los requerimientos del proyecto y no a tendencias del mercado.

---

## Próxima sección

---

En la siguiente sección estudiaremos la búsqueda híbrida, combinando recuperación semántica y búsqueda léxica para mejorar la precisión de sistemas RAG en escenarios empresariales.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

---

## Búsqueda híbrida: combinando significado y coincidencia textual

---

*"Los mejores sistemas RAG no eligen entre búsqueda semántica o búsqueda léxica. Aprovechan las fortalezas de ambas."*

---

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender la diferencia entre búsqueda léxica y búsqueda semántica.
  - Entender el funcionamiento general de BM25.
  - Conocer el concepto de búsqueda híbrida (*Hybrid Search*).
  - Identificar cuándo conviene utilizar cada estrategia dentro de una arquitectura RAG.
-

## Introducción

---

Durante muchos años los motores de búsqueda empresariales se basaron en coincidencias de palabras.

Si un documento contenía exactamente los términos solicitados por el usuario, tenía mayores probabilidades de aparecer entre los primeros resultados.

Este enfoque continúa siendo extremadamente útil.

Sin embargo, presenta limitaciones cuando distintas palabras expresan una misma idea.

La búsqueda semántica resolvió parte de este problema, pero tampoco es perfecta.

La búsqueda híbrida surge para combinar ambos enfoques.

---

## Búsqueda léxica

---

La búsqueda léxica trabaja sobre el texto.

Analiza palabras, frecuencia de aparición y otros indicadores estadísticos.

Uno de los algoritmos más utilizados es **BM25**.

De forma simplificada, BM25 intenta responder una pregunta:

*¿Qué documentos contienen con mayor relevancia los términos utilizados por el usuario?*

Este enfoque resulta especialmente eficaz cuando existen nombres propios, códigos, números de expediente, identificadores o terminología exacta.

---

## Búsqueda semántica

---

La búsqueda semántica trabaja sobre embeddings.

En lugar de comparar palabras, compara significado.

Esto permite recuperar documentos relacionados aunque utilicen vocabularios diferentes.



Por ejemplo, una consulta sobre "licencia por nacimiento" puede recuperar documentación titulada "licencia por paternidad" aunque ninguna palabra coincida exactamente.

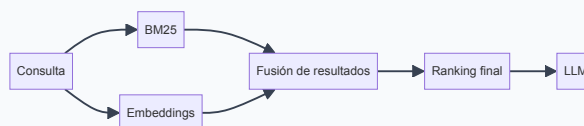
## ¿Por qué combinar ambas?

Cada enfoque resuelve problemas distintos.

La búsqueda léxica destaca cuando los términos exactos son importantes.

La búsqueda semántica sobresale cuando el significado resulta más relevante que la coincidencia textual.

Al combinar ambas se obtiene un sistema más robusto.



## Estrategias de fusión

Existen distintas formas de combinar resultados.

Algunas arquitecturas asignan una puntuación independiente a cada método y luego calculan un ranking conjunto.

Otras aplican primero filtros léxicos y posteriormente una búsqueda semántica sobre el subconjunto resultante.

La estrategia adecuada depende del dominio, del volumen documental y de los objetivos del negocio.

## Caso de estudio

---

Un organismo público dispone de millones de expedientes.

Un usuario consulta un número de resolución específico.

En este escenario, la búsqueda léxica obtiene mejores resultados porque el identificador debe coincidir exactamente.

Otro usuario pregunta:

*"¿Cómo solicitar una prórroga por maternidad?"*

Aquí la búsqueda semántica resulta superior porque puede recuperar documentos relacionados aunque la redacción sea diferente.

La búsqueda híbrida permite responder correctamente en ambos escenarios utilizando una única arquitectura.

---

## Buenas prácticas

---

- Evaluar ambos enfoques con consultas reales.
- No asumir que un único método resolverá todos los casos.
- Ajustar el peso relativo entre búsqueda léxica y semántica según el dominio.
- Medir precisión, cobertura y latencia antes de modificar la estrategia.

---

## Ideas clave

---

- BM25 y los embeddings resuelven problemas diferentes.
  - La búsqueda híbrida aprovecha las fortalezas de ambos enfoques.
  - El ranking final depende de una estrategia de combinación cuidadosamente diseñada.
  - La evaluación debe realizarse sobre datos representativos del negocio.
-

## Próxima sección

---

En la siguiente sección analizaremos el **reranking**, una técnica utilizada para volver a ordenar los documentos recuperados y mejorar aún más la calidad del contexto enviado al modelo de lenguaje.

---

*Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.*

---

## Reranking: mejorando la calidad antes de generar la respuesta

---

*"Recuperar documentos relevantes es importante. Ordenarlos correctamente suele ser aún más importante."*

---

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender qué es el reranking y qué problema resuelve.
  - Diferenciar recuperación inicial de reordenamiento.
  - Conocer el papel de los modelos *cross-encoder*.
  - Incorporar criterios para decidir cuándo el reranking aporta valor.
- 

## Introducción

---

En las secciones anteriores vimos que una consulta se transforma en un embedding y se utiliza para recuperar los documentos más cercanos dentro de una base vectorial.

Sin embargo, la similitud vectorial no siempre produce el orden ideal.

Es frecuente recuperar veinte documentos potencialmente relevantes donde solo tres contienen exactamente la información necesaria.

Enviar todos esos documentos al LLM aumenta el consumo de tokens, incrementa la latencia y puede introducir ruido.

Aquí aparece el reranking.

---

## Dos etapas diferentes

---

Conviene separar claramente ambos procesos.

### Recuperación inicial

Su objetivo es encontrar rápidamente un conjunto reducido de candidatos.

Debe ser veloz incluso sobre millones de documentos.

### Reranking

Su objetivo es analizar con mayor profundidad esos candidatos y ordenarlos según su verdadera relevancia para la consulta.

Al trabajar únicamente sobre unas pocas decenas de documentos puede utilizar modelos más costosos y precisos.

---

## ¿Cómo funciona?

---

El flujo general puede resumirse así:

1. El usuario realiza una consulta.
2. Se recuperan los  $k$  documentos más cercanos mediante búsqueda vectorial o híbrida.
3. Un modelo de reranking compara la consulta con cada documento.
4. Se calcula una nueva puntuación de relevancia.
5. Solo los mejores documentos se envían al LLM.



## ¿Por qué utilizar otro modelo?

Los modelos de embeddings buscan representar significado de forma eficiente.

Los modelos de reranking persiguen un objetivo distinto.

Analizan simultáneamente la consulta y cada documento para estimar con mayor precisión si realmente responden a la necesidad del usuario.

En muchos casos se implementan mediante arquitecturas conocidas como **cross-encoders**, capaces de evaluar relaciones más profundas que una simple distancia entre vectores.

El costo computacional es mayor.

La precisión también.

## ¿Siempre es necesario?

No.

El reranking incorpora complejidad adicional.

En repositorios pequeños o consultas muy específicas puede aportar poco valor.

Sin embargo, cuando existen miles o millones de documentos con contenidos similares suele producir mejoras significativas.

Como en cualquier decisión arquitectónica, la respuesta depende del contexto.

## Caso de estudio

Una compañía mantiene un repositorio con miles de procedimientos de recursos humanos.

Una búsqueda semántica recupera diez documentos relacionados con licencias.

Solo dos describen la licencia por adopción solicitada por el usuario.

El modelo de reranking reordena esos resultados considerando el contenido completo de cada documento y coloca esos dos procedimientos en las primeras posiciones.

El LLM recibe un contexto mucho más preciso y genera una respuesta correctamente fundamentada.

---

## Costos y beneficios

---

Incorporar un paso de reranking implica:

- una llamada adicional a un modelo;
- mayor tiempo de procesamiento;
- consumo extra de recursos.

A cambio puede ofrecer:

- mayor precisión;
- menor cantidad de contexto enviado al LLM;
- reducción del ruido documental;
- mejor aprovechamiento de la ventana de contexto.

El balance debe evaluarse mediante métricas objetivas y pruebas representativas.

---

## Ideas clave

---

- Recuperación y reranking son procesos distintos.
  - El reranking optimiza la calidad del contexto, no la generación.
  - Los modelos de reranking priorizan precisión sobre velocidad.
  - La decisión de incorporarlo debe justificarse con datos y necesidades del negocio.
-

## Próxima sección

---

En la siguiente sección analizaremos cómo evaluar objetivamente un sistema RAG, qué métricas utilizar y cómo medir la calidad de la recuperación y de las respuestas generadas.

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

## Evaluación de sistemas RAG: medir antes de confiar

---

*"No se puede mejorar aquello que no se mide. En un sistema RAG, evaluar es tan importante como recuperar y generar."*

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender por qué evaluar un sistema RAG requiere métricas específicas.
- Diferenciar métricas de recuperación y métricas de generación.
- Conocer indicadores como Recall@K, Precision@K, MRR y nDCG.
- Diseñar una estrategia de evaluación continua para entornos productivos.

## Introducción

---

Un error frecuente consiste en evaluar un sistema RAG únicamente preguntando si "la respuesta parece correcta".

Ese criterio resulta insuficiente.

Una respuesta correcta puede haberse obtenido por casualidad.

Una respuesta incorrecta puede deberse a un problema de recuperación y no del LLM.

Para mejorar un sistema es necesario identificar qué componente está fallando.

Por ello la evaluación debe analizar el pipeline completo.

---

## Dos niveles de evaluación

---

En términos generales conviene separar la evaluación en dos grandes etapas.

### Recuperación

Determina si el sistema encontró los documentos adecuados.

### Generación

Determina si el modelo utilizó correctamente esos documentos para construir la respuesta.

Confundir ambos niveles suele conducir a diagnósticos incorrectos.

---

## Métricas de recuperación

---

### Recall@K

---

Indica si los documentos relevantes aparecen entre los primeros **K** resultados recuperados.

Un Recall@10 elevado significa que el sistema encuentra la información necesaria dentro de los diez primeros documentos.

---

### Precision@K

---

Mide qué proporción de los documentos recuperados realmente resulta relevante.

Una precisión baja implica que el LLM recibirá mucho contexto innecesario.

---

### Mean Reciprocal Rank (MRR)

---

Evalúa la posición del primer documento relevante.



Mientras más arriba aparezca, mayor será el valor de la métrica.

---

## nDCG

---

La métrica **Normalized Discounted Cumulative Gain** considera tanto la relevancia como el orden de los documentos recuperados.

Resulta especialmente útil cuando existen distintos niveles de relevancia.

---

## Evaluando la generación

---

Una recuperación excelente no garantiza una buena respuesta.

También es necesario analizar si el modelo:

- responde la pregunta planteada;
- utiliza correctamente el contexto recuperado;
- evita inventar información;
- cita las fuentes cuando corresponde;
- mantiene coherencia y completitud.

Estas evaluaciones pueden realizarse mediante expertos humanos, conjuntos de referencia o modelos utilizados como evaluadores.

---

## Construyendo un conjunto de pruebas

---

Una buena evaluación requiere consultas representativas.

Conviene incluir:

- preguntas frecuentes;
- consultas ambiguas;
- casos límite;
- preguntas sin respuesta;
- documentos desactualizados;

- terminología específica del dominio.

Cuanto más representativo sea el conjunto de pruebas, mayor confianza ofrecerán los resultados.

---

## Evaluación continua

---

Los sistemas RAG evolucionan constantemente.

Cambian los documentos.

Cambian los modelos.

Cambian las consultas de los usuarios.

Por ello la evaluación no debe ejecutarse una sola vez antes del despliegue.

Debe incorporarse al ciclo de vida del sistema.

Una estrategia habitual consiste en medir automáticamente los indicadores después de cada actualización importante del repositorio documental o del pipeline de recuperación.

---

## Caso de estudio

---

Un equipo reemplaza su modelo de embeddings esperando mejorar la calidad de las respuestas.

Las pruebas muestran que el LLM produce respuestas similares a las anteriores.

Sin embargo, las métricas revelan que el Recall@10 aumentó un 18 % y el MRR mejoró significativamente.

El cambio todavía no resulta evidente para los usuarios, pero la arquitectura recupera mejor la información y ofrece una base más sólida para futuras optimizaciones.

Sin mediciones objetivas, esta mejora habría pasado inadvertida.

---

## Ideas clave

---

- Recuperación y generación deben evaluarse por separado.

- Las métricas permiten localizar el origen de los problemas.
- La evaluación debe utilizar consultas representativas del negocio.
- Medir continuamente forma parte del mantenimiento de un sistema RAG.

---

## Próxima sección

---

En la siguiente sección estudiaremos los principales patrones arquitectónicos para desplegar sistemas RAG en producción, incluyendo escalabilidad, cachés, actualización documental y observabilidad.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

---

## RAG en producción: patrones arquitectónicos para sistemas empresariales

---

---

*"Un prototipo demuestra una idea. Una arquitectura preparada para producción demuestra que esa idea puede sostener un negocio."*

---

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Comprender los principales desafíos de desplegar un sistema RAG en producción.
  - Conocer patrones arquitectónicos utilizados en entornos empresariales.
  - Entender el papel de la observabilidad, las cachés y la actualización documental.
  - Incorporar criterios de diseño orientados a escalabilidad y mantenibilidad.
-

## Introducción

---

Construir un prototipo de RAG suele requerir pocos componentes.

Construir un servicio utilizado por cientos o miles de personas es un desafío completamente distinto.

A medida que aumenta el uso aparecen nuevos requisitos:

- mayor volumen documental;
- usuarios concurrentes;
- restricciones de tiempo de respuesta;
- control de costos;
- auditoría;
- alta disponibilidad;
- seguridad.

La arquitectura debe evolucionar para responder a estas necesidades.

---

## Separación por responsabilidades

---

Una buena práctica consiste en desacoplar los componentes principales del sistema.

En una arquitectura típica encontramos:

- servicio de ingestión documental;
- servicio de generación de embeddings;
- base vectorial;
- servicio de recuperación;
- orquestador RAG;
- Large Language Model;
- servicios de monitoreo y observabilidad.

Cada componente puede escalar y evolucionar de manera independiente.

---

## Actualización del conocimiento

---

El conocimiento corporativo cambia continuamente.

Nuevos procedimientos reemplazan versiones anteriores.

Se incorporan documentos.

Otros dejan de estar vigentes.

Una arquitectura madura debe permitir:

- indexación incremental;
- reindexación completa cuando sea necesario;
- versionado documental;
- eliminación segura de información obsoleta;
- trazabilidad de cada actualización.

Actualizar un repositorio documental no debería requerir detener todo el sistema.

---

## Estrategias de caché

---

Muchas consultas se repiten.

Volver a ejecutar todo el pipeline para cada solicitud puede incrementar costos y latencia.

Dependiendo del dominio pueden incorporarse distintos niveles de caché:

- resultados de búsqueda;
- embeddings de consultas frecuentes;
- respuestas previamente validadas;
- documentos recientemente recuperados.

La caché no reemplaza al RAG.

Complementa su funcionamiento.

---

## Observabilidad

---

Cuando una respuesta resulta incorrecta, el arquitecto necesita saber por qué.

Para ello conviene registrar información como:

- consulta original;
- documentos recuperados;
- puntuaciones de recuperación;
- tiempo consumido en cada etapa;
- modelo utilizado;
- cantidad de tokens;
- costo estimado;
- respuesta final.

Estos registros permiten diagnosticar problemas y mejorar el sistema de forma continua.

---

## Escalabilidad

---

No todos los componentes crecen al mismo ritmo.

La base vectorial puede requerir más memoria.

El servicio de embeddings puede necesitar aceleración mediante GPU.

El LLM puede ejecutarse localmente o consumirse como servicio externo.

Diseñar componentes independientes facilita escalar únicamente aquello que realmente constituye un cuello de botella.

---

## Seguridad y gobierno

---

Los sistemas empresariales deben respetar las mismas políticas que el resto de la organización.

Algunas consideraciones habituales incluyen:

- autenticación y autorización;

- control de acceso por documento;
- cifrado en tránsito y en reposo;
- auditoría de consultas;
- protección de información confidencial;
- cumplimiento normativo.

La búsqueda semántica nunca debe ignorar las reglas de seguridad existentes.

Un usuario solo debería recuperar documentos para los cuales posee autorización.

---

## Arquitectura de referencia

---



Syntax error in text  
mermaid version 10.9.6

Este diagrama resume una arquitectura desacoplada, donde cada componente posee una responsabilidad claramente definida.

---

## Caso de estudio

---

Una empresa implementa un asistente interno para más de dos mil empleados.

Durante las primeras semanas el sistema responde correctamente.

Con el tiempo, la incorporación diaria de nuevos procedimientos comienza a degradar la experiencia porque la indexación solo se ejecuta una vez por semana.

El problema no reside en el LLM ni en la base vectorial.

La arquitectura de ingestión ya no acompaña el ritmo de actualización del negocio.

La solución consiste en incorporar un pipeline incremental que procese únicamente los documentos modificados.

---

## Ideas clave

---

- Un sistema RAG en producción requiere mucho más que un LLM y una base vectorial.
  - La observabilidad es indispensable para diagnosticar problemas.
  - La actualización documental debe formar parte del diseño arquitectónico.
  - Escalabilidad, seguridad y mantenibilidad son requisitos de primer nivel.
- 

## Próxima sección

---

En la última sección del capítulo integraremos todos los conceptos desarrollados y construiremos una visión completa de una arquitectura RAG moderna, preparando el camino para el estudio de agentes de IA y sistemas compuestos.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***

---

## Integrando Retrieval-Augmented Generation: de la teoría a la arquitectura

---

***"RAG no reemplaza al modelo. Lo convierte en un componente de un sistema mucho más amplio."***

---

## Objetivos de aprendizaje

---

Al finalizar esta sección podrás:

- Integrar todos los conceptos desarrollados a lo largo del capítulo.
- Explicar el flujo completo de una arquitectura RAG moderna.
- Identificar las decisiones arquitectónicas críticas en cada etapa.



- Comprender por qué RAG representa un patrón de ingeniería y no únicamente una técnica de IA.
- 

## Introducción

---

En este capítulo recorrimos cada uno de los componentes que conforman un sistema RAG.

Los analizamos por separado para comprender sus responsabilidades.

Sin embargo, en una implementación real todos trabajan de manera coordinada.

La calidad de la solución final depende de esa interacción.

Un excelente LLM no compensará una mala recuperación.

Una base vectorial optimizada no resolverá documentos mal preparados.

Una búsqueda precisa perderá valor si el contexto supera innecesariamente la ventana disponible.

La arquitectura es la que transforma componentes aislados en un sistema útil.

---

## El recorrido completo del conocimiento

---

El ciclo de vida del conocimiento dentro de un sistema RAG puede resumirse en dos grandes procesos.

### Preparación del conocimiento

- incorporación de documentos;
- extracción y normalización;
- chunking;
- enriquecimiento con metadatos;
- generación de embeddings;
- indexación en la base vectorial.

### Respuesta a consultas

- recepción de la pregunta;
- generación del embedding de la consulta;

- recuperación inicial;
- búsqueda híbrida cuando corresponde;
- reranking;
- construcción del contexto;
- generación de la respuesta mediante el LLM.

Cada etapa puede evolucionar independientemente siempre que sus contratos permanezcan estables.

---

## Arquitectura de referencia

---



Syntax error in text  
mermaid version 10.9.6

El diagrama evidencia que el modelo generativo representa únicamente una etapa del proceso.

La mayor parte del trabajo ocurre antes de que el LLM produzca una sola palabra.

---

## Principios arquitectónicos

---

Independientemente de las herramientas elegidas, un sistema RAG sólido suele respetar los siguientes principios.

---

### Separación de responsabilidades

---

Cada componente debe cumplir una función claramente definida.

Esto facilita pruebas, mantenimiento y evolución.

---

### Conocimiento desacoplado

---

Los documentos pertenecen al repositorio documental.

El modelo no debe utilizarse como mecanismo de almacenamiento permanente de conocimiento corporativo.

## Actualización continua

---

El conocimiento cambia.

La arquitectura debe permitir incorporar modificaciones sin reconstruir todo el sistema.

## Observabilidad

---

Cada consulta debe poder reconstruirse posteriormente.

Esto facilita auditorías, análisis de errores y mejora continua.

## Evaluación permanente

---

No basta con desplegar el sistema.

Es necesario medir su comportamiento mediante métricas objetivas.

---

## Errores frecuentes

---

Durante la adopción de RAG aparecen algunos patrones repetitivos.

- Utilizar documentos completos en lugar de fragmentos coherentes.
- Elegir un LLM más grande en lugar de mejorar la recuperación.
- Ignorar los metadatos.
- No evaluar la calidad de la búsqueda.
- Considerar que la indexación es un proceso estático.
- Omitir registros de observabilidad y auditoría.

La mayoría de estos problemas pertenece al diseño arquitectónico y no al modelo.

---

## Caso de estudio

---

Dos organizaciones implementan asistentes internos utilizando exactamente el mismo modelo fundacional.

La primera dedica la mayor parte del esfuerzo a elegir el LLM más potente disponible.

La segunda invierte tiempo en diseñar la ingestión documental, el chunking, los metadatos, la búsqueda híbrida y la evaluación continua.

Meses después ambas comparan resultados.

La diferencia más significativa no proviene del modelo.

Proviene de la arquitectura construida alrededor de él.

Este patrón se repite con frecuencia en proyectos reales.

---

## Lo que viene a continuación

---

Hasta este punto analizamos cómo ampliar el conocimiento de un modelo mediante recuperación documental.

Sin embargo, muchas aplicaciones necesitan algo más que responder preguntas.

Deben tomar decisiones, utilizar herramientas, consultar APIs, ejecutar procesos y coordinar múltiples pasos.

Ese escenario conduce naturalmente al siguiente tema del libro: los **agentes de IA**.

Allí veremos cómo un modelo deja de ser únicamente un generador de texto para convertirse en el núcleo de sistemas capaces de planificar y actuar.

---

## Resumen del capítulo

---

Durante este capítulo aprendimos que:

- RAG separa el conocimiento documental del conocimiento entrenado.
- La preparación de documentos condiciona toda la arquitectura.

- Los embeddings permiten realizar búsqueda semántica.
- Las bases vectoriales optimizan la recuperación por similitud.
- La búsqueda híbrida y el reranking incrementan la precisión.
- La evaluación y la observabilidad son componentes de primer nivel.
- La calidad del sistema depende de la arquitectura completa y no únicamente del LLM.

---

## Mensaje final

---

Retrieval-Augmented Generation representa uno de los cambios más importantes en la Ingeniería de IA moderna.

Demuestra que el valor de un sistema inteligente no reside exclusivamente en el modelo utilizado, sino en la forma en que integra datos, recuperación, generación y gobierno del conocimiento.

Comprender esta arquitectura permite diseñar soluciones escalables, auditables y sostenibles, preparadas para evolucionar junto con las necesidades del negocio.

---

***Un arquitecto no memoriza respuestas. Comprende problemas para poder diseñar soluciones.***